

Documentation API — Reminder Famille

Spécification de tous les endpoints REST exposés par le back-end. Base URL : `http://localhost:3000/api` en développement.

Format : JSON sur l'ensemble de l'API (sauf `/calendar/*.ics` qui renvoie du `text/calendar`).

Authentification : header `Authorization: Bearer <JWT>` sur toutes les routes sauf `/auth/register`, `/auth/login` et `/calendar/:token*`.

Tableau récapitulatif

Méthode	Route	Auth	Description
POST	<code>/auth/register</code>	✗	Création de compte
POST	<code>/auth/login</code>	✗	Connexion
GET	<code>/auth/me</code>	✓	Profil utilisateur courant
POST	<code>/auth/change-password</code>	✓	Changement de mot de passe
PATCH	<code>/users/me</code>	✓	Modification profil
GET	<code>/families</code>	✓	Liste des familles de l'utilisateur
POST	<code>/families</code>	✓	Création d'une famille
POST	<code>/families/join</code>	✓	Rejoindre via code d'invitation
GET	<code>/families/:familyId</code>	✓ + membre	Détail famille + membres
PATCH	<code>/families/:familyId</code>	✓ + admin	Renommer la famille
DELETE	<code>/families/:familyId</code>	✓ + admin	Supprimer la famille
POST	<code>/families/:familyId/leave</code>	✓ + membre	Quitter la famille
POST	<code>/families/:familyId/regenerate-code</code>	✓ + admin	Nouveau code
POST	<code>/families/:familyId/approve/:userId</code>	✓ + admin	Valider un membre pending
PATCH	<code>/families/:familyId/members/:userId</code>	✓ + admin	Modifier rôle/admin
DELETE	<code>/families/:familyId/members/:userId</code>	✓	Retirer un membre
POST	<code>/families/:familyId/members/:userId/reset-password</code>	✓ + admin	Reset MDP membre

GET	/tasks	✓	Liste tâches (avec filtres)
POST	/tasks	✓	Création de tâche
GET	/tasks/:id	✓	Détail d'une tâche
PUT	/tasks/:id	✓	Modification d'une tâche
DELETE	/tasks/:id	✓	Suppression
POST	/tasks/:id/complete	✓	Marquer comme terminée
POST	/tasks/:id/approve	✓ + parent	Valider tâche pending_review
POST	/tasks/:id/reject	✓ + parent	Rejeter tâche pending_review
GET	/tasks/:taskId/comments	✓	Lister commentaires
POST	/tasks/:taskId/comments	✓	Ajouter un commentaire
DELETE	/tasks/:taskId/comments/:commentId	✓	Supprimer un commentaire
GET	/stats/dashboard	✓	Stats du dashboard
GET	/calendar/:token	✗ (auth par token URL)	Flux iCal global
GET	/calendar/:token/perso.ics	✗	Flux iCal perso uniquement
GET	/calendar/:token/family/:familyId.ics	✗	Flux iCal d'une famille

Authentication

POST /api/auth/register

Créer un nouveau compte utilisateur.

Body :

```
{
  "email": "marie@famille.fr",
  "password": "motdepasse123",
  "first_name": "Marie",
  "last_name": "Dupont"
}
```

Réponses :

Code	Cas	Body
201	Compte créé	{ token, user }
400	Champs invalides	{ error, details: { email?, password?, ... } }
409	Email déjà pris	{ error: "Cet email est déjà utilisé" }

Body 201 :

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": 42,
    "email": "marie@famille.fr",
    "first_name": "Marie",
    "last_name": "Dupont",
    "calendar_token": "a3f1...",
    "created_at": "2026-05-28T09:00:00.000Z"
  }
}
```

POST /api/auth/login

Body :

```
{ "email": "marie@famille.fr", "password": "motdepasse123" }
```

Réponses :

- 200 → { token, user }
- 401 → { error: "Identifiants incorrects" }
- 429 → rate limit (5 tentatives / 15min en prod)

GET /api/auth/me

Retourne l'utilisateur courant.

Headers : Authorization: Bearer <JWT>

Réponses :

- 200 → { id, email, first_name, last_name, calendar_token, created_at }
- 401 → token invalide ou expiré



Familles

POST /api/families

Créer une famille (l'utilisateur devient parent + admin).

Body :

```
{ "name": "Famille Dupont" }
```

Réponses :

- 201 → { id, name, invite_code, created_by, created_at }
 - 400 → nom vide ou > 100 caractères
-

POST /api/families/join

Rejoindre une famille via son code (statut initial : pending).

Body :

```
{ "invite_code": "ABC12345" }
```

Réponses :

- 201 → { message: "Demande envoyée...", family_id }
 - 404 → code inexistant
 - 409 → déjà membre (ou demande déjà en cours)
-

POST /api/families/:familyId/approve/:userId

Valider un membre en attente avec attribution du rôle.

Body :

```
{ "role": "child" }
```

(rôle doit être parent ou child)

Réponses :

- 200 → { message: "Membre validé" }
 - 400 → rôle invalide
 - 403 → non-admin
 - 404 → membre introuvable
-

Tâches

GET /api/tasks?category=&status=&family_id=&assigned_to=&from=&to=

Liste des tâches accessibles à l'utilisateur (personnelles + familiales).

Query parameters (tous optionnels) :

- category — filtrer par catégorie
- status — pending , pending_review , completed
- family_id — filtrer par famille
- assigned_to — filtrer par membre assigné

- `from` / `to` — plage de dates (YYYY-MM-DD)

Réponses :

- `200` → `Task[]`

```
[
  {
    "id": 12,
    "user_id": 1,
    "family_id": 5,
    "assigned_to": 3,
    "title": "Sortir poubelle",
    "category": "Corvée",
    "priority": "medium",
    "frequency": "weekly",
    "due_date": "2026-05-29",
    "status": "pending",
    "completed_at": null,
    "completed_by": null,
    "created_at": "2026-05-20T10:00:00.000Z"
  }
]
```

POST `/api/tasks`

Créer une tâche.

Body :

```
{
  "title": "Sortir poubelle",
  "description": "Lundi soir avant 21h",
  "category": "Corvée",
  "priority": "medium",
  "frequency": "weekly",
  "due_date": "2026-05-29",
  "family_id": 5,
  "assigned_to": 3
}
```

Règles :

- Si `family_id` présent → l'utilisateur doit être parent de cette famille (sinon 403)
- Si `family_id` absent → tâche personnelle (toujours autorisée)

Réponses :

- `201` → Task complète
- `400` → champs invalides (titre vide, date mal formée, énum inconnue...)
- `403` → enfant qui tente de créer une tâche familiale

POST /api/tasks/:id/complete

Marquer comme terminée.

Comportement variable selon contexte :

Cas	Effet
Tâche personnelle frequency=once	→ completed
Tâche personnelle récurrente	due_date avancé + reste pending
Tâche familiale completed par parent	idem au-dessus
Tâche familiale completed par enfant assigné	→ pending_review (validation requise)
Tâche familiale completed par enfant non-assigné	403

POST /api/tasks/:id/approve

(parent uniquement, sur tâche pending_review)

Valide la complétion par l'enfant.

- Tâche once → completed
- Tâche récurrente → avancée + reste pending

POST /api/tasks/:id/reject

(parent uniquement, sur tâche pending_review)

Rejette : tâche revient à pending , completed_by réinitialisé.

17

July

Export iCal

GET /api/calendar/:token

Flux iCalendar (RFC 5545) global avec toutes les tâches.

Headers réponse :

Content-Type: text/calendar; charset=utf-8
Content-Disposition: inline; filename="reminder-Marie.ics"

Body 200 (extrait) :

```
BEGIN:VCALENDAR
VERSION:2.0
PRODID:-//Reminder Famille//FR
NAME:Reminder Marie
X-WR-CALNAME:Reminder Marie
BEGIN:VEVENT
UID:task-12@reminder-famille
SUMMARY:Sortir poubelle
```

```
DTSTART;VALUE=DATE:20260529
DESCRIPTION:Lundi soir avant 21h
BEGIN:VALARM
TRIGGER:-P1D
ACTION:DISPLAY
DESCRIPTION:Sortir poubelle
END:VALARM
END:VEVENT
END:VCALENDAR
```

Réponses :

- 200 → flux ICS
- 404 → token inconnu

Authentification spéciale : pas de JWT — le `calendar_token` (48 caractères hexa) sert d'authentification dans l'URL. Sécurité par obscurité acceptable pour ce cas : l'attaquant qui devine un token voit uniquement les tâches d'un utilisateur (read-only, pas de mutation).

GET `/api/calendar/:token/perso.ics`

Sub-feed avec uniquement les tâches personnelles (sans familiales).

GET `/api/calendar/:token/family/:familyId.ics`

Sub-feed avec uniquement les tâches d'une famille spécifique.



Statistiques

GET `/api/stats/dashboard`

Body 200 :

```
{
  "counts": {
    "pending": 5,
    "overdue": 1,
    "pending_review": 2,
    "completed_30d": 12
  },
  "upcoming": [ /* prochaines tâches */ ],
  "families": [ /* breakdown par famille */ ]
}
```

✗ Format des erreurs

Toutes les erreurs respectent le format suivant :

```
{
  "error": "Message lisible en français",
}
```

```
"details": { "champ": "raison" }
}
```

`details` n'est présent que pour les erreurs de validation (400).

Code HTTP	Sens dans cette API
400	Champs invalides ou business rule violée
401	Pas authentifié (pas de token ou token invalide)
403	Authentifié mais action interdite (rôle insuffisant)
404	Ressource introuvable
409	Conflit (ex: email déjà pris)
429	Trop de requêtes (rate limit)
500	Erreur serveur (loggée côté back)

Tester l'API

Avec `curl`

```
# Login
TOKEN=$(curl -sS -X POST http://localhost:3000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"marie@famille.fr","password":"motdepasse123"}' \
  | jq -r .token)

# Liste tâches
curl -sS -H "Authorization: Bearer $TOKEN" \
  http://localhost:3000/api/tasks | jq
```

Suite complète

Le projet fournit `tests/integration.sh` (57 scénarios) qui exécute tous les endpoints avec assertions PASS/FAIL.

```
cd back && npm start &      # démarre le back
bash tests/integration.sh    # → 57/57 passés
```